

Assignment 02 Submission

Name: Emirhan Köksal

Student ID: 20210601040

Git clone command (required format):

```
git clone "https://github.com/koksal100/20210601040_EMIRHAN_KOKSAL.git"  
"20210601040_EMIRHAN_KOKSAL"
```

Part A: Essay

Core Responsibilities of a Linux System Administrator

In the modern IT ecosystem, Linux forms the backbone of cloud computing, web servers, enterprise databases, and DevOps processes. The most critical role that keeps this infrastructure operational is the Linux System Administrator (SysAdmin). A Linux system administrator is responsible for ensuring the health, security, performance, and uninterrupted operation (uptime) of Linux-based systems. This role, which bridges hardware capacity and software operations, requires deep expertise in the operating system kernel, architecture, and command-line interface (CLI). Regardless of scale, a system administrator's core duties can be grouped under four main headings: user management, process control, file system management, and system monitoring.

1. User and Group Management

In Linux, security begins with identity and access management. Because Linux is a multi-user operating system, the system administrator's first task is to determine who can access the system and what privileges they will have. This prevents unauthorized access and protects critical system configurations.

Account Creation and Deletion: System administrators define new users with commands such as `useradd` and `usermod`, and they set home directories and default shells. When an employee leaves the company, the account is securely closed with `userdel`.

Privilege Separation: In accordance with the principle of least privilege, users are grouped according to their roles, such as developer or test specialist, by using `groupadd`. Instead of giving everyone full "root" access, the `/etc/sudoers` file is edited with the `visudo` command so that only specific users are allowed to run limited administrative commands.

Authentication and Security: Setting password expiration periods, enforcing strong password policies, and configuring SSH key-based authentication are part of daily security routines.

2. Process Control and Management

Linux handles tasks as independent execution flows called processes. A system administrator must actively manage these processes in order to optimize resource allocation, maintain system stability, and intervene when applications become unresponsive.

Monitoring System Activity: System administrators use tools such as `ps`, `top`, and `htop` to view CPU and memory consumption in real time and to identify processes that overload the system or stop responding.

Sending Signals and Terminating Processes: When a service stops responding, the administrator sends signals to the process with commands such as `kill`, `pkill`, or `killall`. For example, `SIGTERM` (15) is used for a graceful shutdown, while `SIGKILL` (9) is used to stop a process immediately and forcibly.

Automation and Background Jobs: Background services, also known as daemons, are usually managed through the `systemd` architecture with `systemctl` commands such as `start`, `stop`, `enable`, and `disable`. In addition, `cron` or `systemd` timers are used to automate recurring tasks such as daily backups or log rotations.

3. File System and Storage Management

Data integrity and orderly storage are vital for the sustainability of enterprise applications. Linux administrators are responsible for configuring and maintaining the storage units connected to the operating system.

Partitioning and Formatting: Tools such as `fdisk`, `gdisk`, or `parted` are used to divide physical storage space into logical partitions, while the `mkfs` command is used to apply file systems such as Ext4 or XFS to those partitions.

Logical Volume Management (LVM): LVM, which is widely used in modern systems, allows administrators to

dynamically resize disk space online without needing to shut down the system.

Mounting Operations: Storage areas must be attached to the directory tree before they can be used. Administrators configure the `/etc/fstab` file so that local disks or network shares such as NFS are automatically mounted at system startup.

Permissions and Ownership: To protect sensitive files, read/write/execute permissions are carefully managed with `chmod`, and user/group ownership is carefully managed with `chown`.

4. Monitoring and Performance Optimization

A proactive system administrator does not wait for the system to fail; instead, they continuously monitor the system to detect possible problems before users are affected. System monitoring includes tracking hardware metrics, network performance, and software logs.

Resource Metrics: Disk I/O speeds, network bandwidth, memory, and CPU thresholds are monitored with basic tools such as `vmstat`, `iostat`, and `df` (disk free).

Log Analysis: The `/var/log` directory is the journal of the system. System administrators analyze logs with tools such as `grep` and `tail -f`, or with the `journalctl` command for `systemd` logs, in order to diagnose hardware failures or application errors.

Proactive Alerting Systems: In production environments, centralized monitoring solutions such as Prometheus, Nagios, or Zabbix are integrated so that automatic alerts can be received when metrics move outside safe limits.

Security and Compliance Responsibilities in Linux System Administration

In the modern enterprise world, where information security and legal compliance standards such as KVKK, GDPR, PCI-DSS, and ISO 27001 are critically important, the role of the Linux System Administrator (SysAdmin) has gone far beyond simply keeping systems running. Protecting servers that are targeted by data breaches, ransomware, and cyberattacks is directly the responsibility of the system administrator. A Linux SysAdmin is responsible for proactive security steps such as setting up authentication mechanisms, defining authorization boundaries, and hardening the system against possible threats.

1. User Authentication

The first line of security is verifying that the person attempting to enter the system is who they claim to be. Linux administrators apply modern protocols to minimize cyber risks that may arise from weak authentication methods.

SSH Security and Key-Based Access: In enterprise environments, connecting through SSH with passwords is a major security vulnerability. Administrators completely disable password-based logins and require the use of SSH key pairs (public/private key) instead. Changing the SSH port and preventing the root user from logging in directly through SSH (`PermitRootLogin no`) are also standard procedures.

Multi-Factor Authentication (MFA/2FA): To increase security when accessing critical servers, two-step verification is made mandatory by integrating PAM (Pluggable Authentication Modules) modules that work through Google Authenticator or similar systems.

Centralized Identity Management: In environments with many servers, local password management is not sustainable. SysAdmins integrate centralized authentication systems such as LDAP, Active Directory, or FreeIPA into Linux servers so that users can be controlled from a single center.

2. Authorization

Authorization is the process of determining exactly which resources, such as files, directories, or commands, an authenticated user or service can access within the system.

Advanced File Permissions and ACL: Traditional Linux read/write/execute (rwx) permissions may sometimes be insufficient for complex enterprise needs. Administrators use ACLs (Access Control Lists) to define flexible, user-specific permissions for a particular file or directory independently of traditional group structures, using `getfacl` and `setfacl`.

Managing Sudo Policies: The fully privileged "root" account must always be protected. System administrators configure the `/etc/sudoers` file so that engineers or applications can run only the commands necessary to perform their work, in line with the principle of least privilege.

Mandatory Access Control (MAC): To prevent traditional permissions (DAC) from being bypassed, kernel-level security modules such as SELinux (Security-Enhanced Linux) or AppArmor are kept active. Even if a process is compromised, these modules prevent it from damaging the rest of the system.

3. System Hardening

System hardening is the full set of configuration optimizations performed to minimize a server's attack surface. A Linux distribution installed with its raw default configuration cannot be moved into a production environment without hardening.

Disabling Unnecessary Services and Ports: Every unused service and every open port is a potential backdoor. Administrators scan open ports with `netstat` or `ss`, permanently disable unnecessary services, and remove them when appropriate.

Firewall Management: Strict rules are defined with tools such as `iptables`, `nftables`, `firewalld`, or `ufw` in order to tightly control incoming and outgoing network traffic. Only specific IP addresses or subnets are allowed to access critical services.

Patch Management: The most effective way to close software vulnerabilities is to keep systems up to date. SysAdmins closely follow security patches, such as those applied through `apt-get upgrade` or `dnf update`, and develop automated or planned patching policies that minimize system interruptions.

4. Auditing, Log Management, and Compliance

It is not enough to claim that systems are secure; this must also be provable and auditable within the framework of legal standards.

System Auditing: By setting up the `auditd` infrastructure, also known as the Linux Audit Framework, the system records who made which change to critical system files, such as `/etc/passwd` or password-related files, and exactly when the change occurred.

Centralizing Logs and SIEM Integration: To prevent the risk that logs on the server, such as those under `/var/log`, may be deleted by malicious actors, logs are transferred in real time to a central log server or to a SIEM (Security Information and Event Management) system.

Regular Security Scans: To remain aligned with compliance standards such as PCI-DSS, system administrators perform periodic checks with vulnerability scanning tools such as OpenVAS and Lynis, then update hardening policies according to the resulting reports.

System Maintenance and Automation in Linux System Administration

In growing enterprise infrastructures, it is not enough for systems to be secure; they must also be sustainable, up to date, and stable. A large portion of a Linux System Administrator's (SysAdmin's) time is devoted to routine maintenance operations that keep systems running smoothly and to automation work that removes human error from these operations. Every repetitive task performed manually creates both time loss and error risk. Therefore, a modern Linux administrator places patch management, backup operations, and shell scripting at the center of work processes.

1. Patch and Software Management

In the software world, vulnerabilities and bugs are unavoidable. Regular software updates and patch management are vital for maintaining system stability and reducing cybersecurity risks.

Using Package Managers: Depending on the distribution, package managers such as `apt` for Debian/Ubuntu or `dnf/yum` for RHEL/CentOS/Fedora are used to keep all software on the system up to date.

Security Patches: Administrators can filter and apply only security-focused patches in order to close critical vulnerabilities (CVEs). In production environments, these patches are first tested in test environments to prevent service interruptions.

Live Kernel Patching: On critical servers that cannot tolerate downtime, technologies such as `Kpatch` or Canonical Livepatch are used to close security vulnerabilities in the Linux kernel without rebooting the system.

2. Backup Operations

Data loss is one of the greatest disasters a company can face. Against hardware failures, user mistakes, or cyberattacks, the system administrator must always have a working "Plan B": backups.

Backup Strategies: The industry-standard 3-2-1 backup rule is often applied: 3 copies of the data, on 2 different types of media, with 1 copy stored in a remote location. Full, incremental, and differential backup types are planned according to need.

Basic Backup Tools: While tar and gzip/bzip2 are used to archive files, rsync is preferred for efficiently synchronizing data between local or remote servers because it copies only changed blocks.

Verifying Backups: Taking backups alone is not enough; the system administrator must regularly test whether these backups have become corrupted and must perform restore simulations.

3. Automation Tools and Scripts

A system administrator's efficiency is directly related to how well they can automate. Automating repetitive tasks gives the system administrator the opportunity to focus on more critical architectural problems.

Shell Scripting: Scripts written in Bash or Sh can perform tasks such as checking disk space, automatically creating users, checking service status, and reporting either with a single click or completely unattended.

Scheduled Tasks (Cron Jobs): Through the /etc/crontab file or the crontab -e command, prepared scripts are run automatically at specific times, such as taking backups every night at 03:00 or cleaning logs every Sunday.

Modern Configuration Management: In multi-server environments, instead of writing manual scripts, modern automation tools that manage infrastructure as code, such as Ansible, Puppet, or Chef, can update hundreds of servers from a single center within minutes.

4. The Critical Importance of Log Files

Every event that occurs in the Linux operating system and in the applications running on it is recorded in log files under the /var/log directory. Logs function as the system administrator's "eyes and ears."

Why Are They Important?

Troubleshooting: When a service crashes or the system slows down, the first and most reliable way to find the root cause is to examine journalctl or the /var/log/syslog (messages) files.

Security Auditing: The /var/log/auth.log (secure) file records all successful and failed login attempts to the system. Abnormal activity here makes it possible to detect a potential cyberattack early.

Capacity Planning: Log analysis makes it possible to predict how quickly system resources such as disk and memory are being consumed, allowing hardware upgrades to be planned before the system fails.

Log Rotation (Logrotate): The logrotate service is used to prevent log files from completely filling the disk. This service compresses old logs, archives them, and permanently deletes them after a specified period.

Part B: Make + ask Pipeline

Part B - Pipeline Execution Logic (Make + ask)

In this assignment, an Engineering Action Plan was automatically produced from the codebase.txt file by building a dependency graph (DAG) with GNU Make.

Flow: codebase.txt -> quality.md, perf.md, security.md (parallel) -> quality.sum.md, perf.sum.md, security.sum.md -> concatenated.md -> refined.md -> action.plan.md

Scenario 1: If codebase.txt is updated

- codebase.txt is the root input.
- quality.md, perf.md, and security.md are regenerated in parallel.
- quality.sum.md, perf.sum.md, and security.sum.md are regenerated.
- concatenated.md, refined.md, and action.plan.md are regenerated.
- Result: The plan is rebuilt from the beginning according to the new codebase.

Scenario 2: If security.sum.md is updated

- concatenated.md is regenerated.
- refined.md is regenerated.
- action.plan.md is regenerated.
- The targets on the quality/performance side do not run again.

Scenario 3: If refined.md is updated

- Only action.plan.md is regenerated.
- Upstream files are not run again.

Why make -j?

- Independent targets run at the same time, reducing the total duration.
- Dependent steps wait in the correct order according to the DAG rules.

Makefile Content

```
ASK ?= ./ask
CODEBASE ?= codebase.txt
.PHONY: all clean
all: action.plan.md

# Phase 1 - FAN-OUT (parallel)
quality.md: $(CODEBASE)
    @cat $(CODEBASE) | $(ASK) "You are a senior code reviewer. Analyze ONLY code quality: readability, structure, naming, modularity, duplication, maintainability. Output exactly 5-7 bullet points as 'problem -> fix'. Keep each bullet actionable and concise." > $@

perf.md: $(CODEBASE)
    @cat $(CODEBASE) | $(ASK) "You are a performance engineer. Analyze ONLY performance: bottlenecks, algorithmic inefficiencies, memory/I/O hotspots, unnecessary work. Output exactly 5-7 bullet points as 'issue -> optimization'. Keep each bullet actionable and concise." > $@

security.md: $(CODEBASE)
    @cat $(CODEBASE) | $(ASK) "You are an application security reviewer. Analyze ONLY security: vulnerabilities, unsafe patterns, missing validation, secret handling, auth/authz issues. Output exactly 5-7 bullet points as 'risk -> mitigation'. Keep each bullet actionable and concise." > $@

# Phase 2 - LOCAL SUMMARIZATION
quality.sum.md: quality.md
    @cat quality.md | $(ASK) "Summarize into EXACTLY 5 bullets. Keep only actionable code quality items. Format as '- ...' and keep each bullet short." > $@

perf.sum.md: perf.md
    @cat perf.md | $(ASK) "Summarize into EXACTLY 5 bullets. Keep only actionable performance items. Format as '- ...' and keep each bullet short." > $@

security.sum.md: security.md
    @cat security.md | $(ASK) "Summarize into EXACTLY 5 bullets. Keep only actionable security items. Format as '- ...' and keep each bullet short." > $@

# Phase 3 - CONCAT REPORT (no LLM)
concatenated.md: quality.sum.md perf.sum.md security.sum.md
    @{\
        echo "## Code Quality"; \
        cat quality.sum.md; \
        echo ""; \
        echo "## Performance"; \
        cat perf.sum.md; \
        echo ""; \
        echo "## Security"; \
        cat security.sum.md; \
    } > $@

# Phase 4 - FAN-IN #1 (LLM Refine)
refined.md: concatenated.md
    @cat concatenated.md | $(ASK) "Refine this report by removing duplicates, merging overlaps, and keeping only high-signal actionable issues. Preserve these sections: Code Quality, Performance, Security. Keep concise bullet points." > $@

# Phase 5 - FAN-IN #2 (Final Engineering Action Plan)
action.plan.md: refined.md
    @cat refined.md | $(ASK) "Create a final 'Engineering Action Plan' in markdown. Include prioritized actions (High/Medium/Low), effort estimate for each action (Small/Medium/Large), and recommended execution order. Keep the plan concrete and implementation-oriented." > $@

clean:
    @rm -f quality.md perf.md security.md quality.sum.md perf.sum.md security.sum.md concatenated.md refined.md action.plan.md
```